

Technical report: Exact Maximum Flexi-Clique Computation under Non-Hereditary Feasibility and Connectivity Constraints

A Hardness proof of maximum Flexi-clique problem

A.1 NP-hardness

LEMMA 1. *The Flexi-clique decision problem is NP-hard.*

PROOF. We prove NP-hardness by reduction from the well-known k -clique problem, which is NP-complete in its decision form. Given an instance (G', k) of the clique decision problem, we construct an equivalent instance (G, τ, k) of the Flexi-clique decision problem as follows. We set $G = G'$, meaning that the node and edge sets remain unchanged. We then choose $\tau \in (0, 1)$ sufficiently close to 1 so that for any subset of size x , the degree threshold satisfies $\lfloor x^\tau \rfloor = x - 1$. Intuitively, this setting makes the Flexi-clique condition equivalent to the clique constraint. Formally, it suffices to select τ satisfying

$$\frac{\ln(|V| - 1)}{\ln(|V|)} < \tau < 1,$$

since for any $x < x'$, the ratio $\frac{\ln(x-1)}{\ln(x)} < \frac{\ln(x'-1)}{\ln(x')}$. The parameter k remains the same. If G' contains a clique H of size at least k , then H also forms a valid Flexi-clique in G , because each node has degree $|H| - 1$ within H . Conversely, if G contains a Flexi-clique H of size at least k , the degree constraint forces H to be a clique, implying that G' contains a clique of size at least k . Since the transformation runs in polynomial time, the Flexi-clique decision problem is NP-hard. $\square$

LEMMA 2. *The Maximum Flexi-clique problem is NP-hard.*

PROOF. We prove NP-hardness by reducing from the Flexi-clique decision problem (shown to be NP-hard in Lemma 1). Suppose we have a polynomial-time algorithm $\mathcal{A}$ that finds the maximum Flexi-clique in any instance (G, τ) . Given an instance (G, τ, k) of the k -Flexi-clique problem, we:

- (1) Use $\mathcal{A}$ to compute the largest Flexi-clique $H_{\max}$ and its size $|H_{\max}|$.
- (2) If $|H_{\max}| \geq k$, return yes; otherwise, return no.

Since it solves the NP-hard k -Flexi-clique problem in polynomial time, the Maximum Flexi-clique problem must also be NP-hard. $\square$

A.2 Constant factor inapproximability

THEOREM 1. *The Maximum Flexi-clique problem is constant-factor inapproximable unless $P = NP$.*

PROOF. We prove the claim via a gap-preserving reduction from the gap version of the CLIQUE problem. Specifically, given a graph $G = (V, E)$, an integer K , and a constant $c > 1$, the task is to distinguish between the following two cases, under the promise that one of them holds:

- **YES:** G contains a clique of size at least K ;
- **NO:** every clique in G has size at most K/c .

It is NP-hard to distinguish between the above two cases [1]. Given a graph $G = (V, E)$ with parameters K and $c > 1$, under the promise that G contains a clique of size at least K or that every clique in G has size at most K/c , we construct a graph $G' = (V', E')$ that preserves the promised gap while amplifying differences in clique size. Specifically, for each node $v \in V$, we create a clique C_v consisting of exactly B nodes, where B is a sufficiently large polynomial in n . The node set of G' is defined as $V' = \bigcup_{v \in V} C_v$, and we denote $|V'|$ by N . For any two distinct nodes $u, v \in V$, the cliques C_u and C_v are made fully adjacent if $(u, v) \in E$, and completely non-adjacent otherwise. Next, we set the parameter τ as

$$\tau = 1 - \frac{1}{4n \log N},$$

which will be used to control the gap between s and s^τ in the analysis below. The graph construction process for the gap-preserving reduction is illustrated in Figure 1. For the constructed graph G' , we denote by $\text{OPT}(G')$ the maximum size of a Flexi-clique. **YES Case.** Assume that G contains a clique $Q \subseteq V$ with $|Q| = K$, and let $S = \bigcup_{v \in Q} C_v \subseteq V'$.

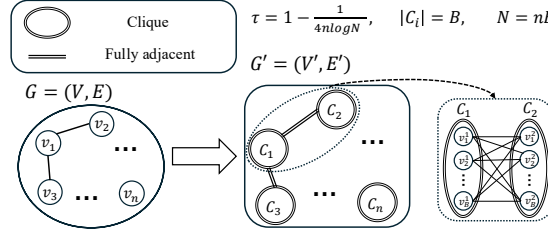


Fig. 1. Reduction process

Clearly, the set S has size KB , and the induced subgraph $G'[S]$ is connected since all cliques corresponding to nodes in Q are adjacent. Consider any node $x \in S$, since C_v is a clique, x has $B - 1$ neighbours inside $G'[C_v]$. Moreover, from the other $K - 1$ cliques in S , the node x has in total $(K - 1)B$ neighbours. Therefore,

$$\deg_{G'[S]}(x) \geq (B - 1) + (K - 1)B = KB - 1. \quad (1)$$

As $\tau \in (0, 1)$, $KB - 1 \geq \lfloor (KB)^\tau \rfloor$ for the chosen value of B . Hence $\deg_{G'[S]}(x) \geq \lfloor |S|^\tau \rfloor$ for all $x \in S$, and thus S is a valid Flexi-clique with $|S| = KB$, implying

$$\text{OPT}(G') \geq KB. \quad (2)$$

NO Case. Now assume that every clique in G has size at most K/c . Let $S \subseteq V'$ be an arbitrary Flexi-clique in G' . For each $v \in V$, define $a_v = |S \cap C_v|$, and let

$$U = \{v \in V : a_v > 0\}. \quad (3)$$

Denote

$$s = |S| = \sum_{v \in U} a_v. \quad (4)$$

We first bound the total number of non-neighbours that a node in a Flexi-clique can have. Recall that $\tau = 1 - \frac{1}{4n \log N}$ and that $s \leq nB = N$. Thus,

$$s - s^\tau \leq \frac{s}{4n} \leq \frac{B}{4}. \quad (5)$$

Fix any $u \in U$ and consider an arbitrary node $x \in S \cap C_u$. By construction of G' , the degree of x in the induced subgraph $G'[S]$ is

$$\deg_{G'[S]}(x) = (a_u - 1) + \sum_{\substack{v \in U: \\ (u,v) \in E}} a_v. \quad (6)$$

Therefore, the number of nodes in S that are non-adjacent to x is

$$(s - 1) - \deg_{G'[S]}(x) = \sum_{\substack{v \in U: \\ (u,v) \notin E}} a_v. \quad (7)$$

Since S is a **Flexi-clique**, we have $\deg_{G'[S]}(x) \geq \lfloor s^\tau \rfloor$, and hence

$$\sum_{\substack{v \in U: \\ (u,v) \notin E}} a_v \leq (s - 1) - \lfloor s^\tau \rfloor \leq s - s^\tau \leq \frac{B}{4}, \quad (8)$$

where the last inequality follows from (5). Now let C be a maximum clique in the induced graph $G[U]$. By the **NO**-case assumption, we have $|C| \leq K/c$. By maximality of C , for every $v \in U \setminus C$ there exists a node $u \in C$ such that $(u, v) \notin E$, and we assign each $v \in U \setminus C$ to one such u . For any fixed $u \in C$, it follows from (8) that

$$\sum_{v \in U \setminus C} a_v \leq \sum_{\substack{v \in U: \\ (u,v) \notin E}} a_v \leq \frac{B}{4}. \quad (9)$$

Summing over all $u \in C$, we obtain

$$\sum_{v \in U \setminus C} a_v \leq |C| \cdot \frac{B}{4}. \quad (10)$$

Since each block C_v has size exactly B , we also have

$$\sum_{u \in C} a_u \leq |C|B. \quad (11)$$

Combining this with (10), we conclude that

$$\begin{aligned} |S| &= \sum_{u \in C} a_u + \sum_{v \in U \setminus C} a_v \\ &\leq |C|B + |C| \cdot \frac{B}{4} = \frac{5}{4}|C|B \leq \frac{5}{4} \cdot \frac{K}{c}B. \end{aligned} \quad (12)$$

Therefore,

$$\text{OPT}(G') \leq \frac{5}{4} \cdot \frac{K}{c}B. \quad (13)$$

In conclusion, combining (2) and (13) yields a constant gap between **YES** and **NO** instances. Therefore, the **Maximum Flexi-clique** problem does not admit any constant-factor approximation unless $P = NP$. $\square$

B Complexity analysis

This section provides the degeneracy-based size bound and a conditional running time below 2^n for **EBA**. Throughout, $\tau \in (0, 1)$ is the parameter and $n = |V|$.

B.1 Size bound from degeneracy

LEMMA 3. *Let ω be the degeneracy of G , that is, the largest minimum degree over all subgraphs of G . Then any Flexi-clique $H \subseteq V$ satisfies $|H| \leq (\omega + 1)^{1/\tau}$.*

PROOF. By the definition of degeneracy, the induced subgraph $G[H]$ contains a node x with $d(x, G[H]) \leq \omega$. Since H is a Flexi-clique, $\lfloor |H|^\tau \rfloor \leq \delta(G[H]) \leq d(x, G[H]) \leq \omega$, hence $|H|^\tau < \omega + 1$, that is, $|H| < (\omega + 1)^{1/\tau}$. $\square$

B.2 A conditional bound below 2^n

Recall that EBA explores a branch-and-bound tree whose search nodes have the form $B = (S, C^r, C^{un}, D)$, and let $\mu(B) = |C^r| + |C^{un}|$ be the number of undecided candidates. In CP-Branching, the child B_i commits the candidates $v_1, \dots, v_i$ to S or D , so it reduces μ by at least i ; ordering the explored children by rank, the j -th of them reduces μ by at least j . While the worst-case running time is $O^*(2^n)$, the following shows that the width-based pivot drives the base strictly below 2 once the branching is kept narrow.

THEOREM 2. *Suppose the width-based pivot bounds the number of explored children at every search node to at most w . Then EBA runs in $O^*(\alpha_w^n)$, where α_w is the largest root of*

$$x^{w+1} - 2x^w + 1 = 0, \quad (14)$$

and $\alpha_w < 2$ for every $w \geq 2$.

PROOF. Let $T(\mu)$ bound the size of the subtree rooted at a search node of measure μ . Since at most w children are explored and the j -th reduces μ by at least j , we have $T(\mu) \leq \sum_{j=1}^w T(\mu - j)$. Substituting $T(\mu) = \alpha^\mu$ and dividing by $\alpha^{\mu-w}$ gives $\alpha^w = \sum_{l=0}^{w-1} \alpha^l = (\alpha^w - 1)/(\alpha - 1)$; multiplying by $(\alpha - 1)$ yields (14). Writing $f(x) = x^{w+1} - 2x^w + 1$, we have $f(1) = 0$ and $f(2) = 2^{w+1} - 2 \cdot 2^w + 1 = 1 > 0$, while f attains a negative local minimum at $x = 2w/(w+1) < 2$ for $w \geq 2$, so its largest root α_w lies in $(1, 2)$. As the per-node work is polynomial in the size of G , $T(n) = O^*(\alpha_w^n)$. $\square$

For instance, $w = 2, 3, 4$ give $\alpha_w \approx 1.618, 1.839, 1.928$, approaching 2 as w grows. This width bound is precisely what the width-based pivot targets. The pivot selects the node p of smallest margin $\bar{d}_{\text{margin}}(p)$, the number of further non-neighbours p can tolerate before violating its degree constraint, and orders the non-neighbours of p first over the whole reachable set. Each admitted non-neighbour raises the in- S non-degree of p by one, so after $\bar{d}_{\text{margin}}(p)$ of them p saturates and every later child, including the include-all child whose partial set then contains all non-neighbours of p , is pruned. The number of explored children is thereby held to about $\bar{d}_{\text{margin}}(p) + 1$, so w is on the order of the pivot's margin. On instances where this margin remains small, the base is well below 2, whereas the sublinear degree requirement of Flexi-clique prevents any constant w from being guaranteed in the worst case.

B.3 Absence of a size threshold for connectivity

The Flexi-clique model imposes only a size-dependent degree constraint and, unlike related dense-subgraph models, provides no size threshold beyond which connectivity is guaranteed. We make this precise below.

PROPERTY 1 (NO SIZE THRESHOLD FOR CONNECTIVITY). *For any $\tau \in (0, 1)$ and any integer h , there exists a node set $H \subseteq V$ with $|H| \geq h$ such that every node $v \in H$ satisfies $d(v, G[H]) \geq \lfloor |H|^\tau \rfloor$, yet $G[H]$ is disconnected.*

PROOF. Fix any $\tau \in (0, 1)$ and any integer h . Choose an integer s such that

$$2s \geq h \quad \text{and} \quad s - 1 \geq \lfloor (2s)^\tau \rfloor. \quad (15)$$

Such an s always exists for sufficiently large s , since $(2s)^\tau$ grows strictly sub-linearly in s for $\tau < 1$. Now construct a graph $G = (V, E)$ consisting of two disjoint cliques C_A and C_B , each of size s , with no edges between them, and let $H = C_A \cup C_B$, so that $|H| = 2s \geq h$. Each node $v \in H$ has degree $s - 1$ in $G[H]$, and by (15) we have $s - 1 \geq \lfloor (2s)^\tau \rfloor = \lfloor |H|^\tau \rfloor$; hence every node satisfies the Flexi-clique degree constraint. However, since there is no edge between C_A and C_B , the induced subgraph $G[H]$ is disconnected. Therefore, for any arbitrarily large size h , there exists a subgraph of size at least h that satisfies the Flexi-clique degree constraint but remains disconnected, so the Flexi-clique model admits no size-based lower bound that guarantees connectivity. $\square$

Consequently, connectivity cannot be inferred from the degree constraint and must be enforced explicitly during the search, which motivates the connectivity-aware branching of EBA.

C Additional experiments

C.1 Empirical branching width

Theorem 2 shows that if the width-based pivot keeps the number of explored children at every search node bounded by a constant w , then EBA runs in $O^*(\alpha_w^n)$ with $\alpha_w < 2$. The bound is conditional: the sublinear degree requirement of Flexi-clique prevents any constant w from being guaranteed in the worst case. We therefore measure the per-node branching width w empirically by instrumenting EBA to record, at every CP-Branching search node, the number of children it actually explores. The key question is whether w stays small relative to the graph size. Two findings show that it does. First, the width does not scale with n : Table 1 reports the width on a fixed LFR family as the number of nodes grows from 10^3 to 10^6 , and the median stays at 2–3 (effective base $\alpha_2=1.618$ to $\alpha_3=1.839$) across four orders of magnitude of graph size. Second, on real networks (Table 2) the width remains a tiny fraction of n —the maximum width is at most a few hundred even on graphs with hundreds of thousands to millions of nodes, i.e. $W_{\max}/n \leq 0.8\%$ and as low as 4×10^{-6} on Florida. What governs the width is the degree structure, not the size: consistent with Lemma 3, w tracks the Flexi-clique (answer) size, which is bounded by the graph’s degeneracy rather than by n . Thus, while the worst-case constant cannot be guaranteed, in practice the branching width stays orders of magnitude below n and does not grow with it, so EBA effectively explores a search tree far narrower than the trivial 2^n bound.

Table 1. Per-node branching width as graph size n grows.

Graph	LFR	LFR	LFR	LFR	Florida
n	10^3	10^4	10^5	10^6	1.07×10^6
W_{med}	3	3	2	2	3
W_{max}	6	17	11	8	4
$\alpha_{W_{\text{med}}}$	1.839	1.839	1.618	1.618	1.839

Table 2. Branching width on real networks ($\tau \in \{0.5, 0.7\}$); $W_{\max}/n$ stays below 1%.

Dataset	AP	CD	CO	GW	DB	FD
n	16,046	39,577	90,303	196,591	317,080	1,070,376
ans.	252	360	255	1443	899	7
W_{med}	44	231	20	255	34	3
$W_{\max}/n$	0.00798	0.00644	0.00194	0.00130	0.00080	3.7×10^{-6}

C.2 Contribution of connectivity-preserving branching

To isolate the value of CP-Branching, we compare it against two exact branching schemes that explore the same search space under identical pruning (the same degree and size rules and the same FPA lower bound) but without connectivity-by-construction: **SE**, a set-enumeration scheme in the style of quasi-clique enumerators that includes each candidate in turn and recurses on the strictly later ones; and **Naive**, binary include/exclude branching. Because both add candidates without regard to adjacency, the partial set may disconnect, so they must verify connectivity explicitly whenever a solution is taken, whereas CP-Branching never does. Table 3 reports the number of explored search-tree nodes; the comparison thus isolates the branching scheme itself. CP-Branching explores dramatically fewer search nodes and is correspondingly faster—on Football at $\tau=0.7$ it finishes after 8198 nodes in 0.11 s, whereas set-enumeration explores 26.4M nodes and takes 30.1 s (280× slower), with the gap widening as graphs grow. **Naive** is competitive on the smallest instances but also blows up (e.g. 191,745 nodes on Football $\tau=0.7$). Across all instances CP-Branching performs zero explicit connectivity checks, since connectivity is preserved by construction. Both the search-node count and the running time confirm that the gain is attributable to the branching scheme rather than to pruning, which is held identical.

Table 3. Branching scheme comparison.

Graph (τ)	Dolphin (0.7)	Railway (0.7)	Football (0.8)	Football (0.7)
CP-Branching	276 / 0.005	212 / 0.011	751 / 0.021	8,198 / 0.11
SE	2,555 / 0.012	48,342 / 0.10	261,789 / 0.33	26.4M / 30.1
Naive	447 / 0.004	6,087 / 0.039	6,275 / 0.034	191,745 / 0.60

Table 4. Branching-order running time

Order	τ	EM	ED	PG	AP	DI	CA	CD	BK	CR	CO	FC	GW	DB	FD	YT
Margin	0.3	0.01	0.04	0.23	6.32	0.29	0.36	8.84	6.78	11.81	5.30	92.01	90.86	104.65	T/O	429.60
	0.5	0.02	0.03	0.04	0.66	0.08	0.12	4.19	1.10	3.03	2.16	51.20	45.61	23.48	1.56	58.09
	0.7	0.03	0.03	0.03	1.05	0.07	0.08	0.34	0.44	1.97	0.35	28.94	3.25	3.72	1.49	T/O
HighDeg	0.3	0.01	0.04	0.28	6.22	0.30	0.38	8.97	7.06	12.05	5.52	93.13	92.88	108.71	T/O	442.17
	0.5	0.02	0.03	0.06	0.81	0.11	0.11	2.12	1.15	2.97	2.25	52.34	23.00	25.84	1.62	63.13
	0.7	0.03	0.03	0.03	1.88	0.07	0.09	0.42	0.54	2.05	0.36	28.54	3.09	4.01	1.70	T/O
Random	0.3	0.01	0.04	0.27	6.20	0.30	0.38	9.02	7.04	12.19	5.58	93.14	98.56	111.58	T/O	444.91
	0.5	0.02	0.03	0.06	0.82	0.09	0.12	2.73	1.15	3.44	1.95	50.21	27.78	23.82	1.60	64.22
	0.7	0.03	0.02	0.03	1.18	0.08	0.09	0.41	0.55	2.04	0.45	28.57	3.08	3.74	1.70	T/O

C.3 Branching-order comparison

The connectivity-preserving scheme of Appendix C.2 fixes which candidates may be branched on; a further degree of freedom is the order in which they are taken. Our default is the margin pivot of Section 5.3, which branches on the least-slack vertex first. Table 4 compares it against a high-degree-first and a random ordering under identical pruning. The three orderings track each other closely on the easy instances, but the margin pivot is the most consistent: it is fastest at $\tau=0.3$ —the hardest setting, where the candidate set is largest—on almost every dataset, and remains the fastest, or within a few percent of the best, at $\tau=0.7$; the orderings are more mixed at $\tau=0.5$. Since the margin pivot never incurs a large penalty and most often yields the lowest running time, **EBA** adopts it as the default branching order.

C.4 Comparison with exact maximum γ -quasi-clique search (IterQC)

The closest exact baseline of EBA is IterQC, a state-of-the-art maximum γ -quasi-clique solver. Both target non-hereditary cohesive models, but the quasi-clique is fundamentally easier: its degree threshold grows linearly with the subgraph size, and for $\gamma \geq 0.5$ a quasi-clique is guaranteed to be connected with diameter at most two, so IterQC can restrict the search to local, bounded-diameter neighbourhoods and never needs an explicit connectivity mechanism. Flexi-clique, by contrast, has a sublinear threshold $\lfloor n^\tau \rfloor$ and requires connectivity explicitly, which is why EBA relies on connectivity-preserving branching. Moreover IterQC does not even apply for $\gamma < 0.5$, whereas EBA solves the full range of τ . We compare EBA (τ) against IterQC (γ) at the matched parameter values 0.5 and 0.7 under a 1-day budget (Tables 5 and 6). Two observations stand out. First, despite solving the harder problem, EBA is competitive and often faster. Second, the maximum Flexi-clique is substantially larger than the maximum quasi-clique at the same parameter, so EBA attains comparable running time while certifying optimality over a much larger and harder search space.

Table 5. Running time (s): EBA vs. IterQC.

Alg.	Param	EM	ED	PG	AP	DI	CA	CD	BK	CR	CO	FC	GW	DB	FD	YT	PK	WK	LJ
EBA	0.5	0.02	0.03	0.05	0.73	0.10	0.11	4.19	1.26	3.23	1.97	45.92	45.08	22.93	1.47	55.75	T/O	T/O	T/O
	0.7	0.03	0.03	0.03	0.98	0.07	0.08	0.29	0.51	1.94	0.30	26.65	2.93	3.15	1.58	T/O	86.31	245.62	439.09
IterQC	0.5	0.04	0.03	2.10	0.52	0.32	1.02	0.55	0.65	T/O	0.26	4.03	1007.24	1.15	1.18	7034.40	271.77	T/O	137.26
	0.7	0.09	0.05	0.18	0.56	0.32	0.83	0.55	0.65	3.08	0.25	32.69	0.98	0.60	1.32	4.38	14.80	T/O	27.06

Table 6. Solution size: maximum Flexi-clique (EBA, τ) vs. maximum quasi-clique (IterQC, γ).

Alg.	Param	EM	ED	PG	AP	DI	CA	CD	BK	CR	CO	FC	GW	DB	FD	YT	PK	WK	LJ
EBA	0.5	440	80	194	993	195	168	360	561	1370	255	9024	1443	899	15	2208	T/O	T/O	T/O
	0.7	137	30	116	252	88	77	93	208	460	87	1846	270	259	7	T/O	244	638	1456
IterQC	0.5	67	21	49	107	51	45	49	101	T/O	45	1051	87	115	7	75	75	T/O	473
	0.7	42	12	42	62	35	28	31	72	82	43	576	51	114	5	39	51	T/O	390

References

- [1] Uriel Feige, Shafi Goldwasser, László Lovász, Shmuel Safra, and Mario Szegedy. 1991. Approximating clique is almost NP-complete. In *[1991] Proceedings 32nd Annual Symposium of Foundations of Computer Science*. IEEE Computer Society, 2–12.